Context-Aware Document Intelligence System

Using a Local Vector Database Management System (Chroma DB)

Course Name	Database Management Systems (DBMS)
Project Type	Vector DBMS & Unstructured Data Retrieval Engine
Student Name / ID	Harsh Oswal
Primary Dataset	The 48 Laws of Power (PDF Document - 476 Pages)
Submission Date	October 16, 2026

1. ABSTRACT & PROBLEM STATEMENT

Traditional Relational Database Management Systems (RDBMS) excel at structured, tabular data queries using exact SQL matches. However, they struggle to index and query unstructured text documents (such as textbooks, research papers, and technical manuals) based on human semantic intent. Standard SQL `LIKE` clauses or full-text keyword indexes fail to recognize contextual synonyms, conceptual relationships, or natural language nuance.

This project resolves this limitation by developing a **Context-Aware Document Intelligence System** powered by a local Vector Database Management System (VDBMS)—**Chroma DB**. Unstructured PDF documents are parsed, split into semantic chunks, and transformed into 384-dimensional vector embeddings using the `all-MiniLM-L6-v2` transformer model. The system constructs a persistent local Hierarchical Navigable Small World (HNSW) vector index to perform approximate nearest neighbor (ANN) similarity searches, returning precise context paragraphs accompanied by page citations.

2 KEY LEARNING OUTCOMES & DBMS CONCEPTS TAUGHT

- **Vector Embeddings:** Mapping high-dimensional semantic meaning into 384-element continuous numerical vectors where distance mirrors semantic similarity.
- **HNSW Graph Indexing:** Understanding sub-linear multi-layer graph structures for rapid Approximate Nearest Neighbor (ANN) search over linear brute-force evaluation.
- **Unstructured Data Architecture:** Managing non-relational document stores through local disk persistence without static database schemas.
- **Hybrid Metadata Mapping:** Coupling high-dimensional vector embeddings with structured scalar metadata (e.g., page numbers and file names) for precise query filtering.
- **Chunking & Context Preservation:** Balancing information retrieval density using overlapping window strategies to preserve sentence boundaries across document splits.

3. COMPLETE STEP-BY-STEP IMPLEMENTATION METHODOLOGY

The project was executed in a structured, modular pipeline transitioning from environment initialization to persistent vector store building and interactive frontend deployment.

476	3,201	384	HNSW
PDF Pages Processed	Vector Chunks Stored	Vector Dimensions	Indexing Algorithm

Phase 1 : Environment Setup & Document Processing

An isolated Python virtual environment (venv) was established to manage dependencies including `chromadb`, `pypdf`, `sentence- transformers`, and `streamlit`. The target PDF (476 pages) was loaded using `PyPDFLoader`. To ensure logical coherence across chunk boundaries, a `RecursiveCharacterTextSplitter` was configured with a window size of 1 ,200 characters and a 1 50-character overlap.

Phase 2: Ingestion & Vector Index Construction (`app. py`)

The chunked document segments were converted into 384-dimensional dense vectors using the `all-MiniLM- L6- v2` transformer model. The chunks, vector representations, auto-generated unique IDs, and page-level metadata were upserted into a persistent local Chroma DB instance at `. /chroma_db`.

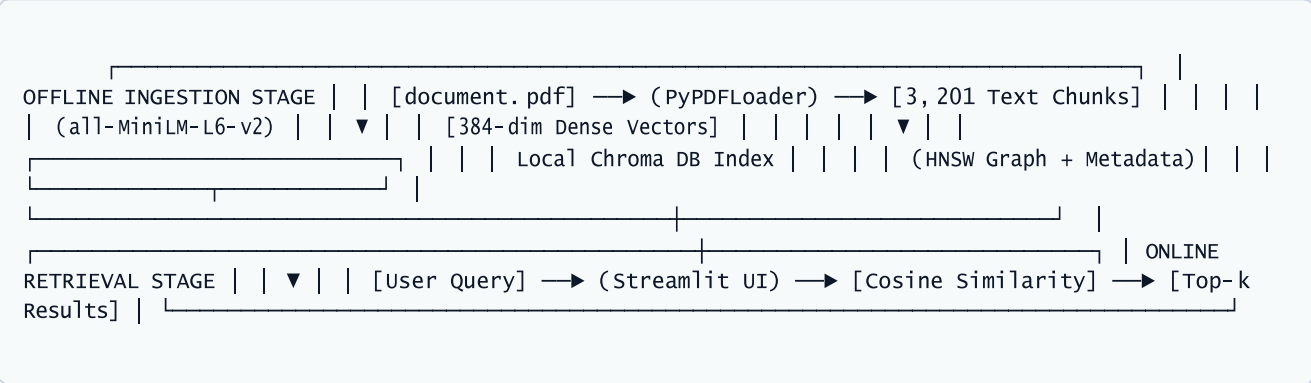
```
      # Core Ingestion Logic (app. py) import chromadb from chromadb. utils import
embedding_functions from langchain_community. document_loaders import PyPDFLoader from
langchain_text_splitters import RecursiveCharacterTextSplitter loader =
PyPDFLoader("document. pdf") docs = loader. load() text_splitter =
RecursiveCharacterTextSplitter(chunk_size=1200, chunk_overlap=150) chunks =
text_splitter. split_documents(docs) chroma_client =
chromadb. PersistentClient( path=". /chroma_db") ef =
embedding_functions. SentenceTransformerEmbeddingFunction(model_name="all- MiniLM- L6- v2")
collection = chroma_client. get_or_create_collection(name="pdf_collection",
embedding_function=ef) collection. add( documents=[c. page_content for c in chunks] , metadatas=
[c. metadata for c in chunks] , ids=[f"id_{i}" for i in range(len(chunks))] )
```

Phase 3: Interactive Dashboard Development (`gui . py`)

A Streamlit web frontend was built to interact with the local persistent database index. Users enter natural language questions, select top-k retrieval depth via a slider, and view retrieved context paragraphs rendered in interactive UI expanders complete with page citations.

4. SYSTEM ARCHITECTURE & COMPONENT WORKFLOW

The system operates via a decoupled architecture separating offline document ingestion and vector indexing from real-time online query retrieval.



5. TECHNICAL STACK SUMMARY

Programming Language	Python 3.10+	Core language runtime and database interface scripts.
Vector Database Engine	Chroma DB (Persistent)	Local vector indexing, HNSW storage, and cosine distance evaluation.
Embedding Transformer	SentenceTransformers	Generates 384-dimensional dense vectors (`all-MiniLM-L6-v2`).
Document Processing	LangChain & PyPDF	PDF text extraction, page tracking, and recursive text chunking.
User Interface (UI)	Streamlit	Web search dashboard with real-time vector retrieval rendering.

6. CURRENT SYSTEM EVALUATION & LIMITATIONS

While the standalone vector database system successfully performs semantic retrieval, it exhibits specific design limitations inherent to pure retrieval engines:

- **Raw Text Return:** Displays verbatim document chunks rather than generating a concise, synthesized answer.
- **Context Fragmentation:** Answers spanning across separate pages remain split across distinct UI expanders.
- **Stateless Queries:** Lacks conversational memory for multi-turn follow-up questions.

7. FUTURE SCOPE & GENERATIVE AI (RAG) INTEGRATION

To overcome current limitations, the system is designed to seamlessly upgrade into a full **Retrieval-Augmented Generation (RAG)** pipeline. By integrating a Large Language Model API (such as Google Gemini API), the retrieved vector database chunks serve as grounded context for automated synthesis.

> **Proposed Enhancement Workflow:**
> User Query ⟶ Chroma DB Vector Search ⟶ Top-k Context Chunks ⟶ Gemini 2.5 Flash API ⟶ Synthesized Natural Language Answer with Page Citations.

Key Advantages of Gemini API Integration:

- **Elimination of Hallucinations:** The LLM is strictly constrained to generate answers using only the retrieved database paragraphs.
- **Context Fusion:** Information from multiple document pages is unified into concise, bulleted summaries.
- **Conversational Memory:** Session state management allows multi-turn dialog (e.g., "Explain point 2 in more detail").

8. CONCLUSION

The **Context-Aware Document Intelligence System** successfully demonstrates the utility of specialized Vector Database Management Systems over traditional relational databases for unstructured text data. By combining recursive text chunking, 384-dimensional vector embeddings, and persistent local HNSW graph indexing in Chroma DB, the system achieves fast, accurate, and contextually aware semantic searches over a 476-page textbook. The project establishes a production-ready retrieval foundation that scales seamlessly into advanced Generative AI and RAG architectures.

9. REFERENCES

1. Taipalus, T. (2024). *Vector database management systems: Fundamental concepts, use-cases, and current challenges*. Cognitive Systems Research, 85, 1 01 21 6.
2. Malkov, Y. A., & Yashunin, D. A. (2020). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. IEEE TPAMI.
3. Chroma DB Documentation. *Local Persistent Vector Store Architecture*. https://docs.trychroma.com/